SEMINAR REPORT

On

QML UNLEASHED: SUPERPOSITION & ENTANGLEMENT



by

Pratiksha Kumari

Examination Roll Number: 251H0660073

MCA – III Semester

Department of Computer Science

Banasthali Vidyapith, Rajasthan – 304022

Session: 2026–2027

TABLE OF CONTENTS

Abstract

1 Introduction . . . 1
2 Quantum Machine Learning . . . 1
2.1 Introduction to Quantum Machine Learning . . . 1
2.2 Fundamental Principles of QML . . . 1
2.3 Objectives of Quantum Machine Learning . . . 2
2.4 Classification of QML Approaches . . . 2
2.5 Challenges and Future Scope . . . 3
3 Classical and Quantum Bits . . . 3
3.1 Classical Bits . . . 3
3.2 Quantum Bits (Qubits) . . . 3
3.3 Quantum Properties . . . 4
3.3.1 Superposition . . . 4
3.3.2 Entanglement . . . 4
3.3.3 Measurement . . . 5
3.4 Quantum Computation and Algorithms . . . 5
4 Quantum Principles . . . 6
4.1 Superposition Principle . . . 6
4.2 Quantum Entanglement . . . 6
4.2.1 Role of Entanglement in Quantum Machine Learning . . . 7
4.3 Quantum Interference . . . 7
4.4 Summary of Quantum Principles . . . 8
5 Quantum Gates and Circuits . . . 8
5.1 Structure of a Quantum Circuit . . . 8
5.2 Hadamard Gate . . . 8
5.3 Pauli Gates . . . 9
5.3.1 Pauli-X Gate . . . 9
5.3.2 Pauli-Y Gate . . . 9
5.3.3 Pauli-Z Gate . . . 10
5.4 Controlled-NOT (CNOT) Gate . . . 10
5.5 Rotation Gates . . . 11
5.6 Applications of Quantum Gates in QML . . . 12
6 Fundamental Quantum States . . . 12
6.1 Bell States: Definition and Fundamental Properties . . . 12
6.2 Quantum Gates Used to Generate a Bell State . . . 12
6.2.1 Hadamard Gate . . . 13
6.2.2 Controlled-NOT (CNOT) Gate . . . 13
6.3 Generation of the Bell State $|\Phi^+\rangle$ . . . 13

6.4 Greenberger-Horne-Zeilinger (GHZ) States 14
6.4.1 Generation of a GHZ State 14
6.4.2 Properties and Applications of GHZ States 15
7 Data Encoding Strategies in Quantum Machine Learning 15
7.1 Basis Encoding 15
7.2 Amplitude Encoding 16
7.3 Angle Encoding 16
7.4 Feature Map Encoding 17
7.5 Comparison of Data Encoding Methods 17
8 Qiskit Implementation of Superposition and Entanglement 18
8.1 Implementation of Superposition 18
8.1.1 Python Implementation 18
8.1.2 Result and Explanation 19
8.2 Implementation of Entanglement 19
8.2.1 Python Implementation 19
8.2.2 Result and Explanation 20
8.3 Comparison of Superposition and Entanglement 21
9 Overview of Quantum Machine Learning Algorithms 21
9.1 Variational Quantum Classifier 21
9.2 Quantum Support Vector Machine 22
9.3 Quantum Neural Networks 22
9.4 Quantum Approximate Optimization Algorithm 23
9.5 Quantum Generative Models 23
9.6 Comparison of QML Algorithms 24
9.7 Importance of Superposition and Entanglement in QML 24

Abstract

QML Unleashed: Superposition & Entanglement discusses the concepts of superposition and entanglement, which are fundamental principles of quantum computing, and their implications for Quantum Machine Learning (QML). A classical computer stores information in bits, whereas a quantum computer uses quantum bits, or qubits, which can exist in a combination of quantum states, a phenomenon known as superposition. Entanglement allows two or more qubits to share quantum correlations that cannot be described independently.

This seminar explores superposition and entanglement through their implementation in quantum circuits and examines the unique phenomena associated with them. Quantum gates such as the Hadamard, Pauli-X, and CNOT gates are used to demonstrate these concepts. Bell states and GHZ states illustrate important quantum correlations. The seminar also discusses data encoding methods used in QML, including basis encoding, angle encoding, and amplitude encoding.

Using Qiskit, these principles can be explored through the implementation and simulation of quantum circuits. The report further introduces QML models and algorithms such as Quantum Neural Networks (QNNs) and Variational Quantum Classifiers (VQCs). Finally, it highlights potential applications of superposition and entanglement in QML, along with the opportunities and challenges of quantum computing in machine learning.

Keywords: Quantum Machine Learning (QML), Superposition, Entanglement, Qubits, Data Encoding, Quantum Gates, Bell States, GHZ States, Quantum Neural Networks (QNNs), Variational Quantum Classifiers (VQCs).

1 Introduction

Quantum computing is an emerging field of computer science that uses the principles of quantum mechanics to process information. Unlike classical computers, which use bits represented as either 0 or 1, quantum computers use quantum bits (qubits), which can exist in a superposition of states.

Two important concepts in quantum computing are superposition and entanglement. Superposition allows a qubit to exist in a combination of the states $|0\rangle$ and $|1\rangle$, while entanglement creates correlations between qubits that cannot be explained by classical physics alone.

Quantum Machine Learning (QML) combines quantum computing with machine learning techniques. It explores how quantum systems can be used to represent data, perform computations, and develop new approaches to learning problems.

This seminar focuses on the fundamental concepts of quantum computing, the differences between classical and quantum bits, quantum gates and circuits, fundamental quantum states, and data encoding strategies. It also introduces the implementation of quantum circuits using Qiskit and provides an overview of important Quantum Machine Learning algorithms.

The main objective of this seminar is to understand the role of superposition and entanglement in quantum computing and explore their potential applications in Quantum Machine Learning.

2 Quantum Machine Learning

2.1 Introduction to Quantum Machine Learning

Quantum computing has emerged as a transformative computational paradigm that exploits the fundamental principles of quantum mechanics to address computational problems. Quantum algorithms have demonstrated the potential for computational speedups in specific tasks, attracting significant attention in areas such as cryptography, optimization, and scientific simulation . Meanwhile, the rapid growth of Artificial Intelligence (AI) and Machine Learning (ML) has led to increasingly complex models that require substantial computational resources for processing high-dimensional data .

Quantum Machine Learning (QML) combines quantum computing with machine learning by incorporating quantum mechanical principles, including **superposition, entanglement, and quantum interference**, into learning algorithms. This field explores whether quantum systems can improve data representation, computational efficiency, and learning capabilities.

Several QML approaches have been proposed, including Quantum Neural Networks (QNNs), Quantum Convolutional Neural Networks (QCNNs), and hybrid quantum–classical architectures. These approaches combine quantum circuits with classical optimization techniques to develop new computational models .

2.2 Fundamental Principles of QML

The development of QML relies on several important quantum computing principles:

- **Superposition:** A qubit can exist in a quantum state that is a combination of the basis states $|0\rangle$ and $|1\rangle$.
- **Entanglement:** Entanglement creates quantum correlations between qubits that cannot be described as independent quantum states.
- **Quantum Interference:** Quantum interference allows probability amplitudes to reinforce or cancel one another, influencing measurement outcomes.
- **Quantum Parallelism:** Quantum operations can act on superpositions of states. However, measurement does not reveal all computational outcomes simultaneously.

These principles form the theoretical foundation for designing quantum circuits that can be integrated into machine learning workflows.

2.3 Objectives of Quantum Machine Learning

The main objectives of studying Quantum Machine Learning are:

- **Review Existing Approaches:** Study major QML models, including QNNs, QCNNs, and hybrid quantum–classical learning architectures.
- **Analyze Model Characteristics:** Understand the computational cost, expressive capabilities, and scalability of quantum and classical learning methods.
- **Identify Challenges:** Examine challenges related to quantum noise, optimization, limited hardware resources, and model evaluation.
- **Explore Future Applications:** Investigate possible research directions and practical applications of QML in real-world problems.

2.4 Classification of QML Approaches

For systematic study, QML approaches can be classified according to four major dimensions.

1. **Data Encoding Strategy:**

   Data encoding describes how classical information is represented in quantum states. Common approaches include basis encoding, angle encoding, and amplitude encoding. The choice of encoding influences circuit complexity, resource requirements, and the representation of data.

2. **Quantum Circuit Architecture:**

   Quantum circuit architecture defines the structure of a quantum learning model. Examples include Variational Quantum Circuits (VQCs), Quantum Neural Networks (QNNs), Quantum Convolutional Neural Networks (QCNNs), and hybrid quantum–classical architectures.

3. **Optimization and Training Strategy:**

   QML models may use gradient-based optimization, parameter shift rules, quantum natural gradient methods, and hybrid optimization procedures. Training strategies influence convergence, computational cost, and sensitivity to noise.

4. **Application Domain:**

   QML research investigates applications in computer vision, healthcare, cybersecurity, physics simulation, finance, and optimization. The practical suitability of each approach depends on the problem, available data, and quantum hardware capabilities.

2.5 Challenges and Future Scope

Despite its potential, practical QML remains challenging. Current quantum processors are affected by noise, limited qubit counts, and restrictions on circuit depth. These factors can reduce computational reliability and make large-scale training difficult . Furthermore, the absence of consistent benchmarking procedures can complicate comparisons between quantum and classical models.

Future research will focus on improving quantum circuit design, developing effective training methods, reducing hardware noise, and establishing reliable evaluation frameworks. Continued research is required to determine the specific problems for which QML can provide practical advantages over classical machine learning.

“‘latex

3 Classical and Quantum Bits

All computing systems rely on the ability to store and manipulate information. Classical computers process information using bits, which represent data as either binary 0 or 1. In contrast, quantum computers use quantum bits, commonly known as qubits, to process information according to the principles of quantum mechanics.

A classical bit can exist in only one of two states, 0 or 1, at a time. A qubit, however, can exist in a superposition of the basis states $|0\rangle$ and $|1\rangle$. Quantum computers exploit properties such as superposition, entanglement, and interference to perform certain computational tasks in ways that differ from classical computing.

Qubits can be implemented using different physical systems, including superconducting circuits, trapped ions, and photons. Superconducting qubits require extremely low temperatures to maintain their quantum states and reduce errors caused by environmental disturbances. For this reason, superconducting quantum processors are commonly operated at temperatures close to absolute zero.

3.1 Classical Bits

A classical bit is the basic unit of information in a classical computing system. It has two possible values: 0 and 1. All classical data, including numbers, text, images, and instructions, can be represented using combinations of these binary values.

Classical logic gates manipulate bits through operations such as AND, OR, and NOT. These operations form the foundation of conventional computer processors.

3.2 Quantum Bits (Qubits)

A quantum bit, or qubit, is the basic unit of information in a quantum computing system. Unlike a classical bit, a qubit can exist in a superposition of the states $|0\rangle$ and $|1\rangle$. Its general pure state can be expressed as

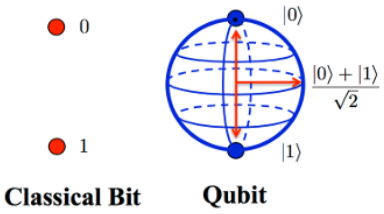


Figure 1: Classical Bit and Quantum bit

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where α and β are complex probability amplitudes satisfying the normalization condition

$$|\alpha|^2 + |\beta|^2 = 1.$$

When the qubit is measured in the computational basis, the probability of obtaining 0 is $|\alpha|^2$, while the probability of obtaining 1 is $|\beta|^2$.

Qubits can be realized through several physical technologies. One important implementation uses superconducting circuits, which must be cooled to extremely low temperatures to protect their quantum states from environmental noise and disturbances.

3.3 Quantum Properties

Three fundamental concepts that play an important role in quantum computing are superposition, entanglement, and measurement.

3.3.1 Superposition

Superposition is a quantum property in which a qubit can be described as a combination of the basis states $|0\rangle$ and $|1\rangle$. This allows quantum systems to represent probability amplitudes associated with multiple possible measurement outcomes.

For example, applying a Hadamard gate to a qubit initialized in the state $|0\rangle$ produces the state

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

When measured in the computational basis, this state produces 0 or 1 with equal probability. Superposition is an important resource for quantum algorithms, although it does not mean that all possible answers can be directly read from a single measurement.

3.3.2 Entanglement

Entanglement is a quantum phenomenon in which two or more qubits share a joint state that cannot be represented as a product of independent states. Measurements on entangled qubits can exhibit correlations that cannot be explained by classical local models.

For example, a two-qubit Bell state can be represented as

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

When both qubits are measured in the computational basis, the outcomes are perfectly correlated: the results are either 00 or 11, each with probability one-half. Entanglement is important in quantum communication, quantum information processing, and several quantum algorithms.

3.3.3 Measurement

Measurement is the process used to obtain classical information from a quantum system. When a qubit in a superposition is measured in the computational basis, the outcome is either 0 or 1, with probabilities determined by its state amplitudes.

For the state

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

the measurement probabilities are

$$P(0) = |\alpha|^2, \qquad P(1) = |\beta|^2.$$

Measurement therefore converts quantum information into a classical result. The outcome of a single measurement is probabilistic, and repeated measurements can be used to estimate the probability distribution of the quantum state.

3.4 Quantum Computation and Algorithms

A quantum computer can perform many computational tasks that are also possible on classical computers. However, simply executing a classical algorithm on a quantum processor does not automatically provide a speed advantage. To benefit from quantum computing, algorithms must exploit quantum properties in an appropriate way.

Quantum algorithms use quantum gates, interference, and entanglement to manipulate quantum states and obtain useful measurement outcomes. Their performance depends on the problem being solved and the computational resources required.

One well-known example is Shor's algorithm, introduced by Peter Shor in 1994. It provides an efficient quantum approach to integer factorization compared with the best-known classical methods for sufficiently large inputs. This has important implications for public-key cryptographic systems that rely on the difficulty of factorization.

Quantum computers are also being investigated for molecular simulation and optimization. For example, quantum algorithms can help estimate molecular energy levels, while variational quantum algorithms combine parameterized quantum circuits with classical optimization to search for low-energy states. Such approaches are active research areas, and their practical performance depends on the available quantum hardware.

Current quantum processors face challenges such as noise, limited qubit counts, and restricted circuit depth. Future fault-tolerant quantum computers aim to use quantum error correction to perform longer computations more reliably.

4 Quantum Principles

Quantum computing is based on fundamental principles of quantum mechanics that distinguish it from classical computing. The most important principles include superposition, entanglement, and quantum interference. These principles enable quantum systems to represent and manipulate information in ways that differ from classical computers. In Quantum Machine Learning (QML), they are used to construct quantum circuits, represent complex data relationships, and develop quantum learning models.

4.1 Superposition Principle

One of the major principles of quantum computing is superposition. It allows a quantum system to exist in a combination of quantum states until a measurement is performed. Unlike a classical bit, which has a definite value of either 0 or 1, a qubit can be represented as a linear combination of the computational basis states.

The general state of a single qubit is expressed as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where α and β are complex probability amplitudes. They satisfy the normalization condition:

$$|\alpha|^2 + |\beta|^2 = 1$$

The probability of measuring the state $|0\rangle$ is $|\alpha|^2$, whereas the probability of measuring $|1\rangle$ is $|\beta|^2$.

Superposition does not mean that a quantum computer directly reveals all possible answers simultaneously. Instead, quantum algorithms manipulate probability amplitudes using quantum gates and interference to increase the probability of obtaining useful results.

In Quantum Machine Learning, superposition enables quantum circuits to transform input data into quantum states that may represent complex features. Such representations are being investigated for classification, optimization, and pattern recognition tasks.

4.2 Quantum Entanglement

Quantum entanglement is another fundamental principle of quantum mechanics. It occurs when two or more qubits share a joint quantum state that cannot be expressed as a product of independent states. Consequently, the system must be described as a whole rather than as separate qubits.

A well-known example is the Bell state:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}\left(|00\rangle + |11\rangle\right)$$

In this state, measurements of both qubits in the computational basis produce correlated outcomes. The possible results are 00 and 11, each occurring with probability $1/2$.

Entanglement differs from ordinary classical correlation because quantum correlations cannot always be explained by classical local models. However, entanglement does not allow information to be transmitted instantaneously between distant particles.

Entanglement has important applications in quantum communication, quantum teleportation, quantum cryptography, and quantum error correction. It is also investigated in quantum simulation, optimization, and machine learning.

4.2.1 Role of Entanglement in Quantum Machine Learning

In Quantum Machine Learning, entangling operations are commonly used in Quantum Neural Networks (QNNs) and Variational Quantum Circuits (VQCs). These operations create correlations between qubits and allow quantum models to represent interactions among input features.

Entangled quantum circuits can help QML models to:

- Represent complex relationships between input features.
- Explore high-dimensional quantum state spaces.
- Support expressive quantum representations for classification tasks.
- Investigate optimization problems using quantum circuits.

However, excessive entanglement and deeper circuits can increase implementation complexity and sensitivity to hardware noise. Therefore, selecting an appropriate entanglement structure is an important consideration when designing QML models.

4.3 Quantum Interference

Quantum interference is the phenomenon in which probability amplitudes combine constructively or destructively, affecting the probabilities of measurement outcomes. It plays an important role in quantum algorithms because quantum gates can change both the magnitudes and the relative phases of probability amplitudes.

Constructive interference occurs when probability amplitudes combine to increase the amplitude associated with a particular outcome.

Destructive interference occurs when probability amplitudes partially or completely cancel one another, reducing the amplitude associated with an outcome.

Quantum algorithms use interference to manipulate probability distributions and increase the likelihood of obtaining desired results. This requires carefully designed sequences of quantum operations rather than simply evaluating every possible answer independently.

For example, Grover's search algorithm uses amplitude amplification to increase the probability of measuring a desired item in an unsorted search space. Quantum optimization algorithms also use quantum operations to explore candidate solutions, although their performance depends on the algorithm and the problem being solved.

- Transforming encoded input data through quantum circuits.
- Modifying measurement probabilities during model training.
- Supporting classification and regression models.
- Exploring quantum optimization and learning strategies.

The effectiveness of these methods depends on circuit architecture, data encoding, optimization procedures, and hardware noise.

4.4 Summary of Quantum Principles

Superposition, entanglement, and quantum interference are interrelated principles that form the foundation of quantum computation. Superposition represents combinations of basis states, entanglement creates non-classical correlations between qubits, and interference changes probability amplitudes through quantum operations.

In Quantum Machine Learning, these principles provide tools for constructing quantum feature representations and parameterized circuits. Research continues to investigate when and how these quantum properties can provide practical benefits over classical machine learning methods.

5 Quantum Gates and Circuits

Quantum information is manipulated and processed through quantum gates and quantum circuits. Similar to classical logical gates, quantum gates operate on quantum states, but they follow the principles of quantum mechanics. Mathematically, a quantum gate is represented by a unitary operator that transforms one or more qubits while preserving the total probability.

A quantum circuit consists of a sequence of quantum gates applied to qubits. The input qubits are prepared in an initial state, transformed through quantum operations, and measured to obtain classical information. The sequence and arrangement of gates directly influence the output and computational capabilities of a quantum algorithm.

In Quantum Machine Learning (QML), quantum circuits are used to encode data, transform quantum states, and implement trainable learning models. Parameterized quantum circuits are particularly important because their gate parameters can be optimized during training, similar to the trainable parameters of classical neural networks.

5.1 Structure of a Quantum Circuit

A quantum circuit generally consists of three main stages:

1. **State Preparation:** Qubits are initialized into a suitable quantum state. Classical input data may be encoded into these states using appropriate data encoding techniques.

2. **Quantum Transformation:** Quantum gates are applied to manipulate the qubits. Single-qubit gates transform individual qubits, whereas multi-qubit gates establish interactions between qubits.

3. **Measurement:** The quantum states are measured to obtain classical outcomes, which can then be processed by conventional computational methods.

Parameterized quantum circuits contain adjustable gate parameters that can be optimized using classical optimization algorithms. These circuits form an important component of variational quantum algorithms and quantum neural networks.

5.2 Hadamard Gate

The Hadamard gate is a fundamental single-qubit gate that creates a superposition of computational basis states. It is commonly used to initialize qubits into balanced superposition states.

The matrix representation of the Hadamard gate is:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

When applied to the state $|0\rangle$, it produces:

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

Similarly, when applied to $|1\rangle$, it produces:

$$H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

The Hadamard gate is important in quantum algorithms because it creates superposition and enables interference effects. In Quantum Machine Learning, it can be used during circuit initialization and as part of quantum data-processing operations.

5.3 Pauli Gates

Pauli gates are fundamental single-qubit gates represented by the Pauli matrices. The three principal Pauli gates are the Pauli-X, Pauli-Y, and Pauli-Z gates. They perform different transformations on quantum states and are widely used in quantum circuits.

5.3.1 Pauli-X Gate

The Pauli-X gate is equivalent to the classical NOT operation when applied to computational basis states. It exchanges the states $|0\rangle$ and $|1\rangle$.

Its matrix representation is:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

The transformations are:

$$X|0\rangle = |1\rangle$$

$$X|1\rangle = |0\rangle$$

The Pauli-X gate is used for bit-flip operations, state preparation, and quantum circuit manipulation.

5.3.2 Pauli-Y Gate

The Pauli-Y gate performs a quantum state transformation that combines a bit flip with a phase change. Its matrix representation is:

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

Its action on the computational basis states is:

$$Y|0\rangle = i|1\rangle$$

$$Y|1\rangle = -i|0\rangle$$

The Pauli-Y gate is used in quantum state manipulation and in constructing quantum operations involving changes in both state and phase.

5.3.3 Pauli-Z Gate

The Pauli-Z gate changes the phase of the state $|1\rangle$ while leaving the state $|0\rangle$ unchanged.

Its matrix representation is:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

The transformations are:

$$Z|0\rangle = |0\rangle$$

$$Z|1\rangle = -|1\rangle$$

The Pauli-Z gate is important in phase manipulation, quantum algorithms, and quantum error correction. Pauli gates can also be incorporated into quantum machine learning circuits to construct different quantum state transformations.

5.4 Controlled-NOT (CNOT) Gate

The Controlled-NOT gate, commonly known as the CNOT gate, is a two-qubit quantum gate used to establish interactions between qubits. It consists of a control qubit and a target qubit.

The target qubit remains unchanged when the control qubit is $|0\rangle$. When the control qubit is $|1\rangle$, the target qubit is flipped.

The CNOT gate performs the following transformations:

$$|00\rangle \rightarrow |00\rangle$$

$$|01\rangle \rightarrow |01\rangle$$

$$|10\rangle \rightarrow |11\rangle$$

$$|11\rangle \rightarrow |10\rangle$$

The matrix representation of the CNOT gate, using the basis ordering $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, is:

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

The CNOT gate is particularly important because it can create entanglement when applied to a suitable superposition state. For example, applying a Hadamard gate to the first qubit followed by a CNOT gate produces a Bell state:

$$|00\rangle \xrightarrow{H\otimes I} \frac{|00\rangle + |10\rangle}{\sqrt{2}}$$
$$\xrightarrow{\text{CNOT}} \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

CNOT gates are widely used in quantum algorithms, variational quantum circuits, and quantum neural networks to create correlations between qubits.

5.5 Rotation Gates

Rotation gates are parameterized quantum gates that rotate a qubit state around different axes of the Bloch sphere. They are particularly important in Quantum Machine Learning because their parameters can be adjusted during the training process.

The three principal rotation gates are:

- Rotation around the X-axis: $R_x(\theta)$
- Rotation around the Y-axis: $R_y(\theta)$
- Rotation around the Z-axis: $R_z(\theta)$

The rotation around the X-axis is represented by:

$$R_x(\theta) = \begin{bmatrix} \cos(\theta/2) & -i\sin(\theta/2) \\ -i\sin(\theta/2) & \cos(\theta/2) \end{bmatrix}$$

The rotation around the Y-axis is:

$$R_y(\theta) = \begin{bmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{bmatrix}$$

The rotation around the Z-axis is:

$$R_z(\theta) = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}$$

Here, θ represents the rotation angle. By adjusting this parameter, the quantum circuit can produce different quantum state transformations.

In QML, rotation gates serve as trainable components of parameterized quantum circuits. Classical optimization algorithms adjust their parameters to minimize a loss function or improve the model's predictions.

Rotation gates are commonly used in:

- Variational quantum algorithms.
- Quantum neural networks.
- Quantum classification models.
- Quantum optimization algorithms.

5.6 Applications of Quantum Gates in QML

Quantum gates form the basic building blocks of quantum machine learning models. Different gates perform different functions: Hadamard gates generate superposition, Pauli gates transform quantum states, CNOT gates establish entanglement, and rotation gates provide adjustable parameters for learning.

By combining these gates into suitable circuit architectures, QML models can encode input data, learn quantum state transformations, and produce measurement outcomes for classification, regression, and optimization tasks.

The performance of these models depends on the circuit architecture, data encoding strategy, training method, and the noise characteristics of the quantum hardware.

6 Fundamental Quantum States

Quantum states describe the condition of a quantum system. In quantum computing, qubits can exist in superposition and can become entangled through quantum gates. These properties are important for quantum information processing and quantum machine learning.

6.1 Bell States: Definition and Fundamental Properties

A two-qubit quantum state is called entangled if it cannot be expressed as a tensor product of two individual single-qubit states. Entanglement produces quantum correlations and is an important resource in quantum computing, quantum communication, and quantum cryptography.

The four Bell states form an orthonormal basis for a two-qubit system. They are defined as

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}\left(|00\rangle + |11\rangle\right)$$

$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}\left(|00\rangle - |11\rangle\right)$$

$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}\left(|01\rangle + |10\rangle\right)$$

$$|\Psi^-\rangle = \frac{1}{\sqrt{2}}\left(|01\rangle - |10\rangle\right).$$

These states are mutually orthogonal and normalized. Their orthonormality can be expressed as

$$\langle B_i|B_j\rangle = \delta_{ij},$$

where $|B_i\rangle$ and $|B_j\rangle$ represent Bell states, and δ_{ij} is the Kronecker delta.

Bell states are useful in quantum teleportation, superdense coding, and entanglement-based quantum communication.

6.2 Quantum Gates Used to Generate a Bell State

Two quantum gates are required to generate the Bell state $|\Phi^+\rangle$: the Hadamard gate and the Controlled-NOT (CNOT) gate.

6.2.1 Hadamard Gate

The Hadamard gate creates a superposition from a computational basis state. Its matrix representation is

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

When applied to the state $|0\rangle$, it produces

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle).$$

6.2.2 Controlled-NOT (CNOT) Gate

The CNOT gate operates on two qubits: a control qubit and a target qubit. It flips the target qubit only when the control qubit is in state $|1\rangle$.

Its matrix representation, using the basis order $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, is

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

Its action on the computational basis states is

$$\text{CNOT}|00\rangle = |00\rangle, \qquad \text{CNOT}|01\rangle = |01\rangle,$$

$$\text{CNOT}|10\rangle = |11\rangle, \qquad \text{CNOT}|11\rangle = |10\rangle.$$

6.3 Generation of the Bell State $|\Phi^+\rangle$

A Bell state can be generated by starting with two qubits in the initial state $|00\rangle$, applying a Hadamard gate to the first qubit, and then applying a CNOT gate with the first qubit as the control and the second as the target.

Step 1: Initial State

The initial state of the two-qubit system is

$$|\psi_0\rangle = |00\rangle.$$

Step 2: Apply the Hadamard Gate

Applying the Hadamard gate to the first qubit gives

$$|\psi_1\rangle = (H \otimes I)|00\rangle = \frac{1}{\sqrt{2}} \left(|00\rangle + |10\rangle\right),$$

where I represents the identity operation on the second qubit.

Step 3: Apply the CNOT Gate

The CNOT gate leaves $|00\rangle$ unchanged and transforms $|10\rangle$ into $|11\rangle$. Therefore,

$$|\psi_f\rangle = \text{CNOT}(H \otimes I)|00\rangle.$$

The final state is

$$|\psi_f\rangle = \frac{1}{\sqrt{2}}\left(|00\rangle + |11\rangle\right) = |\Phi^+\rangle.$$

The resulting state is entangled and cannot be written as a product of two individual qubit states. When measured in the computational basis, the outcomes 00 and 11 each occur with probability $1/2$ in an ideal experiment. This demonstrates the perfect correlation between the two measurement outcomes.

The Bell state illustrates how the Hadamard gate creates superposition and the CNOT gate produces entanglement. These operations form a fundamental example of quantum circuit construction and are useful for understanding quantum information processing and quantum machine learning.

6.4 Greenberger-Horne-Zeilinger (GHZ) States

Greenberger-Horne-Zeilinger (GHZ) states are important examples of multipartite entanglement in quantum computing. While a Bell state represents entanglement between two qubits, a GHZ state extends this concept to three or more qubits. These states demonstrate quantum correlations among multiple qubits and are useful for studying entanglement, quantum information processing, and the performance of quantum computers.

An N-qubit GHZ state is defined as

$$|\mathrm{GHZ}_N\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle^{\otimes N} + |1\rangle^{\otimes N}\right),$$

where N represents the number of qubits. The notation $|0\rangle^{\otimes N}$ represents all qubits in state $|0\rangle$, while $|1\rangle^{\otimes N}$ represents all qubits in state $|1\rangle$.

For three qubits, the GHZ state is written as

$$|\mathrm{GHZ}_3\rangle = \frac{1}{\sqrt{2}}\left(|000\rangle + |111\rangle\right).$$

6.4.1 Generation of a GHZ State

A GHZ state can be generated using a Hadamard gate and multiple CNOT gates. First, all qubits are initialized in the state $|0\rangle$. A Hadamard gate is applied to the first qubit to create a superposition. Then, CNOT gates are applied sequentially, using the first qubit as the control and the remaining qubits as targets.

For a three-qubit system, the initial state is

$$|\psi_0\rangle = |000\rangle.$$

After applying a Hadamard gate to the first qubit, the state becomes

$$|\psi_1\rangle = \frac{1}{\sqrt{2}}\left(|000\rangle + |100\rangle\right).$$

Next, a CNOT gate is applied from the first qubit to the second qubit, followed by another CNOT gate from the first qubit to the third qubit. The final state is

$$|\psi_f\rangle = \frac{1}{\sqrt{2}}\left(|000\rangle + |111\rangle\right) = |\mathrm{GHZ}_3\rangle.$$

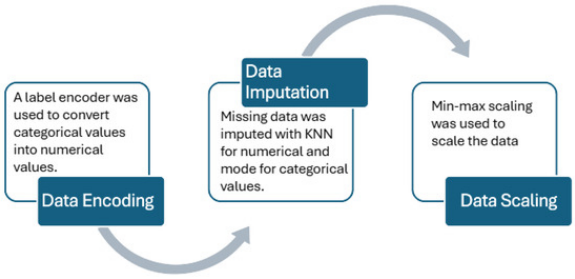


Figure 2: Encoding

6.4.2 Properties and Applications of GHZ States

GHZ states exhibit genuine multipartite entanglement, meaning that the complete state cannot be described as a mixture of states that are separable across different partitions of the qubits. When measured in the computational basis, an ideal GHZ state produces either all zeros or all ones, with each outcome occurring with probability $1/2$.

GHZ states are useful for investigating quantum correlations, testing quantum devices, studying the effects of noise and decoherence, and exploring quantum error correction. As the number of qubits increases, preparing and maintaining GHZ states becomes more challenging because quantum gates are imperfect and interactions with the environment can destroy quantum coherence.

Therefore, GHZ states provide an important way to understand multipartite entanglement and evaluate the capabilities of quantum computing systems.

7 Data Encoding Strategies in Quantum Machine Learning

Data encoding is an important step in Quantum Machine Learning (QML). It involves transforming classical data into quantum states so that quantum circuits can process the information. Unlike classical machine learning, which represents data using conventional numerical values, QML uses qubits and quantum states to represent and process information.

The choice of encoding technique affects the number of qubits required, circuit complexity, and the performance of a QML model. Different encoding methods are suitable for different types of data and learning tasks.

7.1 Basis Encoding

Basis encoding represents classical binary data directly using computational basis states of qubits. In this method, each classical bit is mapped to a corresponding qubit state.

For example, the classical binary string 1001 can be encoded as

$$|x\rangle = |1001\rangle.$$

Here, four qubits represent the four bits of the input string. Each qubit is initialized to either $|0\rangle$ or $|1\rangle$, depending on the corresponding classical bit.

Basis encoding is simple and useful for binary data. However, the number of qubits required increases with the length of the input string.

7.2 Amplitude Encoding

Amplitude encoding represents classical data using the probability amplitudes of a quantum state. A normalized classical vector is mapped to the amplitudes of a quantum state.

For a classical vector

$$\mathbf{x} = \begin{bmatrix} x_0 & x_1 & \cdots & x_{N-1} \end{bmatrix}^T,$$

the corresponding quantum state is

$$|\psi_{\mathbf{x}}\rangle = \sum_{i=0}^{N-1} x_i |i\rangle,$$

where the amplitudes satisfy the normalization condition

$$\sum_{i=0}^{N-1} |x_i|^2 = 1.$$

For example, a normalized vector with four elements can be represented using two qubits because two qubits provide four computational basis states.

Amplitude encoding can represent large vectors using relatively few qubits. However, preparing the required quantum state can be computationally demanding, depending on the data and the encoding procedure.

7.3 Angle Encoding

Angle encoding maps classical numerical features to rotation angles of quantum gates. It is commonly used in parameterized quantum circuits and variational quantum machine learning models.

For a feature vector

$$\mathbf{x} = (x_1, x_2, \ldots, x_N),$$

each feature can be encoded using a rotation gate. For example, using a rotation about the y-axis,

$$R_y(x_i) = \begin{bmatrix} \cos(x_i/2) & -\sin(x_i/2) \\ \sin(x_i/2) & \cos(x_i/2) \end{bmatrix}.$$

When applied to the initial state $|0\rangle$, the gate produces

$$R_y(x_i)|0\rangle = \cos(x_i/2)|0\rangle + \sin(x_i/2)|1\rangle.$$

Thus, the value of a classical feature determines the rotation angle and consequently the resulting quantum state.

Angle encoding is relatively straightforward to implement. However, when the dataset contains many features, multiple qubits or repeated encoding operations may be required.

7.4 Feature Map Encoding

Feature map encoding transforms classical input data into quantum states so that similarities and relationships between data points can be investigated using quantum operations.

A quantum feature map can be represented as

$$\phi : \mathcal{X} \to |\phi(x)\rangle,$$

where x is a classical input and $|\phi(x)\rangle$ is its corresponding quantum state.

The similarity between two encoded inputs can be evaluated using the inner product

$$K(x, z) = |\langle\phi(x)|\phi(z)\rangle|^2.$$

Here, $K(x, z)$ represents a quantum kernel that measures the similarity between the quantum states associated with inputs x and z.

Quantum feature maps are particularly useful in quantum kernel methods and quantum support vector machines. They allow classical data to be represented in a quantum state space, where quantum circuits can capture relationships between features.

7.5 Comparison of Data Encoding Methods

The main encoding techniques differ in the way they represent classical information and in their implementation requirements.

Table 1: Comparison of quantum data encoding methods

Encoding Method	**Representation**	**Main Characteristic**
Basis Encoding	Computational basis states	Simple representation of binary data.
Amplitude Encoding	Probability amplitudes	Can represent large vectors with relatively few qubits.
Angle Encoding	Quantum gate rotation angles	Suitable for numerical features and parameterized circuits.
Time-Evolution Encoding	Hamiltonian-driven evolution	Encodes data through quantum dynamics.
Hamiltonian Encoding	Hamiltonian operators	Useful in quantum simulation and certain matrix algorithms.
Feature Map Encoding	Quantum feature states	Useful for kernel methods and measuring data similarity.

Data encoding establishes the connection between classical datasets and quantum computation. Basis encoding is useful for binary data, amplitude encoding represents information through quantum amplitudes, and angle encoding uses numerical features to control quantum gate rotations. Time-evolution and Hamiltonian encoding connect data with quantum dynamics, while feature map encoding supports similarity calculations in quantum machine learning.

Selecting an appropriate encoding technique is important because it influences circuit complexity, state preparation, the number of required qubits, and the overall efficiency of a QML model.

8 Qiskit Implementation of Superposition and Entanglement

Qiskit is an open-source framework used to create, simulate, and analyze quantum circuits. It provides a practical way to understand fundamental quantum computing concepts such as superposition and entanglement through programming.

In this section, Qiskit is used to demonstrate superposition using the Hadamard gate and entanglement using the Controlled-NOT (CNOT) gate. The circuits are executed on a quantum simulator, and their measurement results are analyzed using histograms.

8.1 Implementation of Superposition

Superposition is a fundamental property of quantum mechanics in which a qubit can exist in a combination of the computational basis states $|0\rangle$ and $|1\rangle$.

A qubit initially in state $|0\rangle$ can be placed into an equal superposition by applying the Hadamard gate:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle).$$

The resulting state has equal probabilities of being measured as 0 or 1.

8.1.1 Python Implementation

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

# Create a one-qubit circuit
qc = QuantumCircuit(1, 1)

# Apply Hadamard gate to create superposition
qc.h(0)

# Measure the qubit
qc.measure(0, 0)

# Display the circuit
print(qc.draw())

# Simulate the circuit
simulator = AerSimulator()
result = simulator.run(qc, shots=1000).result()
```

```
# Display measurement counts
counts = result.get_counts()
print(counts)

# Plot the results
plot_histogram(counts)
plt.show()
```

8.1.2 Result and Explanation

The Hadamard gate transforms the initial state $|0\rangle$ into an equal superposition of $|0\rangle$ and $|1\rangle$. After 1000 measurement shots, the counts for outcomes 0 and 1 should be approximately equal.

$$P(0) = \frac{1}{2}, \qquad P(1) = \frac{1}{2}.$$

The exact counts may vary between runs. This experiment demonstrates how a single quantum gate can create superposition in a qubit.

8.2 Implementation of Entanglement

Quantum entanglement occurs when two or more qubits form a joint quantum state that cannot be expressed as a product of independent states. Entanglement can be generated by combining superposition with a two-qubit gate such as CNOT.

Starting with the two-qubit state $|00\rangle$, a Hadamard gate is applied to the first qubit, followed by a CNOT gate using the first qubit as the control and the second as the target.

The resulting state is

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle).$$

This is a Bell state, which is an example of a maximally entangled two-qubit state.

8.2.1 Python Implementation

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

# Create a two-qubit circuit
qc = QuantumCircuit(2, 2)

# Create superposition on the first qubit
qc.h(0)

# Apply CNOT to entangle the qubits
qc.cx(0, 1)
```

```
# Measure both qubits
qc.measure([0, 1], [0, 1])

# Display the circuit
print(qc.draw())

# Simulate the circuit
simulator = AerSimulator()
result = simulator.run(qc, shots=1000).result()

# Display measurement counts
counts = result.get_counts()
print(counts)

# Plot the results
plot_histogram(counts)
plt.show()
```

8.2.2 Result and Explanation

The Hadamard gate creates a superposition in the first qubit. The CNOT gate then correlates the two qubits, producing the entangled state

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle).$$

When measured in the computational basis, the expected outcomes are 00 and 11, each with a probability of 50%. The outcomes 01 and 10 have zero probability in an ideal simulation.

This result demonstrates the correlation between the two qubits. Although the measurement histogram illustrates this correlation, the histogram alone does not prove entanglement for an arbitrary quantum state.

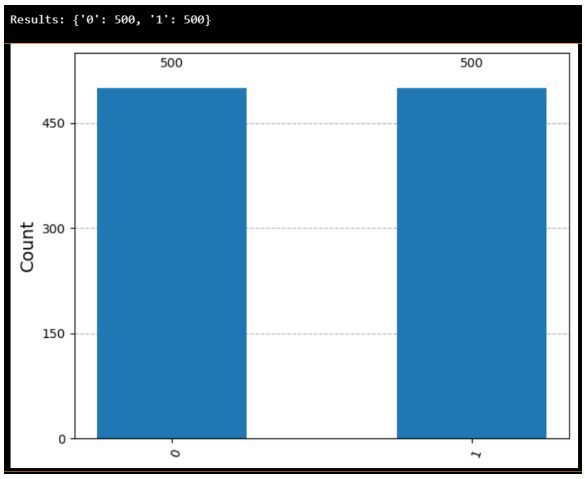


Figure 3: Superposition

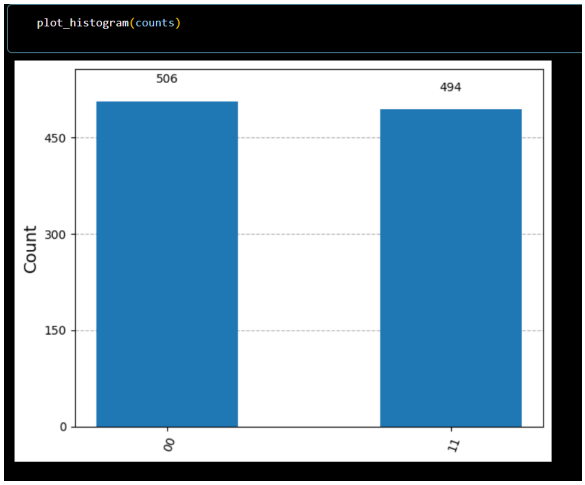


Figure 4: Qiskit simulation results illustrating quantum entanglement

8.3 Comparison of Superposition and Entanglement

Table 2: Comparison of Superposition and Entanglement

Feature	**Superposition**	**Entanglement**
Meaning	A qubit exists in a combination of basis states.	Qubits form a joint state that cannot be separated into individual states.
Gates Used	Hadamard gate	Hadamard and CNOT gates
Qubits Required	Can be demonstrated with one qubit.	Requires at least two qubits.
Example	$\frac{\|0\rangle+\|1\rangle}{\sqrt{2}}$	$\frac{\|00\rangle+\|11\rangle}{\sqrt{2}}$

9 Overview of Quantum Machine Learning Algorithms

Quantum Machine Learning (QML) combines the principles of quantum computing with machine learning techniques. It uses quantum states, quantum gates, and measurement operations to process information and develop models for tasks such as classification, regression, clustering, and data analysis.

Two important concepts in QML are superposition and entanglement. Superposition allows a qubit to exist in a combination of computational basis states, whereas entanglement creates quantum correlations between multiple qubits. These properties are used in different ways depending on the algorithm and circuit architecture. However, they do not automatically make every quantum machine learning algorithm faster or more accurate than its classical counterpart.

9.1 Variational Quantum Classifier

A Variational Quantum Classifier (VQC) is a hybrid quantum-classical algorithm used for classification tasks. It encodes classical input data into a quantum circuit and applies parameterized quantum gates to learn patterns in the data.

Role of Superposition: Hadamard or rotation gates can create superposition in the qubits. This allows the circuit to represent combinations of basis states while processing the input data.

Role of Entanglement: Entangling gates, such as CNOT, can establish correlations between qubits. These correlations may help the circuit represent relationships among different input features.

Applications:

- Binary and multiclass classification
- Pattern recognition
- Small-scale scientific and experimental datasets

9.2 Quantum Support Vector Machine

A Quantum Support Vector Machine (QSVM) uses quantum circuits to map classical data into quantum feature spaces. A quantum feature map transforms an input x into a quantum state $|\phi(x)\rangle$. A quantum kernel can then be estimated using the overlap between two encoded states:

$$K(x, z) = |\langle\phi(x)|\phi(z)\rangle|^2.$$

Role of Superposition: Superposition can help represent encoded data across multiple basis states, depending on the chosen feature map.

Role of Entanglement: Entangling gates can encode interactions between input features, potentially producing feature representations that are difficult to reproduce with simple, independent encodings.

Applications:

- Classification
- Feature-space analysis
- Experimental quantum-kernel learning

9.3 Quantum Neural Networks

Quantum Neural Networks (QNNs) use parameterized quantum circuits as trainable models. Their parameters are adjusted by a classical optimizer to reduce a defined loss function.

Role of Superposition: Superposition enables the circuit to operate on quantum states containing contributions from multiple basis states.

Role of Entanglement: Entanglement allows qubits to share non-classical correlations. In a QNN, entangling layers can help represent dependencies between features and qubits.

Applications:

- Classification and regression experiments
- Quantum state learning
- Hybrid quantum-classical model research

9.4 Quantum Approximate Optimization Algorithm

The Quantum Approximate Optimization Algorithm (QAOA) is primarily an optimization algorithm rather than a general-purpose machine learning algorithm. It uses parameterized quantum circuits to search for approximate solutions to discrete optimization problems.

Role of Superposition: An initial superposition can represent a combination of candidate bit-string solutions.

Role of Entanglement: Problem-dependent operations can introduce entanglement between qubits associated with interacting variables. This can influence the probability distribution over candidate solutions.

Applications:

- Combinatorial optimization
- Scheduling and resource allocation research
- Optimization subproblems relevant to machine learning

9.5 Quantum Generative Models

Quantum generative models aim to learn a data distribution and generate samples that approximate it. Examples include quantum circuit Born machines and quantum generative adversarial models.

Role of Superposition: Superposition allows a quantum circuit to assign probability amplitudes to multiple possible measurement outcomes.

Role of Entanglement: Entanglement can represent correlations among different parts of the generated data distribution.

Applications:

- Experimental data generation
- Modeling complex probability distributions
- Research into generative machine learning

9.6 Comparison of QML Algorithms

Table 3: Role of Superposition and Entanglement in QML Algorithms

Algorithm	**Use of Superposition**	**Use of Entanglement**
Variational Quantum Classifier	Represents combinations of basis states in the circuit.	Models correlations between qubits and encoded features.
Quantum Support Vector Machine	Supports quantum feature-state representations.	Can encode interactions between input features.
Quantum Neural Network	Provides quantum-state representations for trainable circuits.	Can capture correlations between qubits.
QAOA	Represents candidate solutions in a superposition.	Can correlate variables involved in the optimization problem.
Quantum Generative Models	Assigns amplitudes to possible measurement outcomes.	Can represent dependencies in generated data.

9.7 Importance of Superposition and Entanglement in QML

Superposition and entanglement are useful resources in quantum machine learning, but their roles are different. Superposition provides a quantum representation involving multiple basis states, while entanglement allows joint states to contain correlations that cannot be described as independent qubit states.

In practical QML circuits, superposition may be created using Hadamard or rotation gates, while entanglement is often introduced using CNOT or other two-qubit gates. The benefit depends on the dataset, encoding method, circuit depth, noise, and training procedure. Therefore, the presence of superposition and entanglement alone does not guarantee a computational advantage.

Bibliography

[1] N. Mishra, M. Kapil, H. Rakesh, et al., "Quantum machine learning: A comprehensive survey of models, challenges, and future directions," *ScienceDirect*, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0304397526001829

[2] G. Rajitha, B. Maheshkumar, P. Rajendar, and P. K. D, "Quantum Machine Learning: Principles, Algorithms, Applications, And Research Challenges," *International Journal of Innovative Research in Technology (IJIRT)*, vol. 13, no. 3, pp. 787–808, Aug. 2026. [Online]. Available: https://ijirt.org/publishedpaper/IJIRT207401_PAPER.pdf

[3] K. Zaman, A. Marchisio, M. A. Hanif, and M. Shafique, "A Survey on Quantum Machine Learning: Basics, Current Trends, Challenges, Opportunities, and the Road Ahead," *arXiv preprint arXiv:2310.10315*, 2025. [Online]. Available: https://arxiv.org/abs/2310.10315

[4] S. Pattanayak, *Quantum Machine Learning with Python: Using Cirq from Google Research and IBM Qiskit*, 1st ed. Apress, 2021.